

Relatório de Uso de Inteligência Artificial

Projeto: Explorador Urbano — Rastreamento GPS com Ranking Multiusuário

1. Contexto da Atividade

A atividade proposta consistia em criar, a partir de um código base fornecido em aula, uma aplicação que utilizasse rastro de localização GPS. O modelo de IA utilizado em todo o processo foi o **Claude (Anthropic)**, acessado via interface web `claude.ai`. A interação se deu em linguagem natural, com envio de arquivos e documentos de contexto em cada etapa do desenvolvimento.

2. Contexto Inicial Fornecido à IA

Antes de qualquer instrução de código, foi fornecido à IA um **documento PDF de proposta de projeto** (*proposta-projeto.pdf*) com bastante contexto sobre a ideia e a direção técnica desejada. O documento cobria:

- **Ideia geral:** construir um rastreador de exploração urbana inspirado no app CityStrides, que coloriza no mapa cada rua já percorrida pelo usuário, com foco na cidade de Pelotas (centro histórico e campus UFPel).
- **Fluxo de uso:** o usuário inicia o rastreo no celular, o navegador envia coordenadas GPS via POST ao servidor, o backend calcula quais segmentos de rua foram percorridos usando dados do OpenStreetMap, e o mapa exibe ao vivo o trajeto atual e todo o histórico colorido em azul.
- **Componentes técnicos detalhados:** endpoint POST `/location`, transmissão via SSE (Server-Sent Events), persistência em SQLite, consulta à API Overpass (OpenStreetMap) para map-matching, captura via `navigator.geolocation.watchPosition()`, renderização com `L.Polyline` e painel de estatísticas.
- **Justificativa técnica:** extensão direta do código base (Projeto Rei Julian), rodando localmente com `go run`, sem infraestrutura externa.

Esse documento serviu como ponto de partida rico: a IA recebeu não apenas o que deveria ser feito, mas o *porquê*, o público-alvo, o contexto geográfico e as escolhas tecnológicas já pensadas pelo aluno. Isso permitiu que as respostas geradas fossem muito mais alinhadas ao objetivo real do projeto.

3. Etapas de Uso da IA e Prompts Aplicados

Etapa 1 — Envio da proposta e análise do código base

Com a proposta já enviada como contexto, o arquivo ZIP do projeto base (*trabalho_explorador.zip*) foi enviado com instrução de **não refazer do zero**, mas de entender a arquitetura existente (Flask + SQLite + Leaflet) e propor melhorias mantendo compatibilidade total com o código original.

Etapa 2 — Especificação das melhorias desejadas

Em uma mensagem estruturada foram solicitadas 10 categorias de mudanças: sistema de usuários com login local, associação de sessões a usuários, aba de Ranking diário (distância, tempo, pace, velocidade), ranking ao vivo com status *andando/parado/pausado*, WebSocket via Flask-SocketIO, novas tabelas no banco (*users*, *online_status*), cálculo automático de pace e calorias, melhorias visuais com abas e medalhas para top 3, manutenção do mapa/GPS e explicação prévia antes de qualquer código.

Etapa 3 — Implementação gerada pela IA

A IA gerou o plano completo (tabelas, endpoints, mudanças de frontend) antes de escrever qualquer código. Em seguida produziu o **main.py** reestruturado e o **index.html** completo com login, 4 abas (Mapa, Meu Histórico, Ranking, Ao Vivo), Socket.IO, Chart.js e medalhas ■■■■.

Etapa 4 — Revisão e detecção de bugs

O arquivo final (*explorador-final.zip*) foi enviado com pedido de análise sem modificações. A IA identificou 5 problemas: chave incompatível *elapsed_fmt* vs *elapsed_formatted* (bug real), dois `setInterval` sobrepostos, função JS morta com lógica errada, dependência desnecessária e `PRAGMA WAL` repetido.

Etapa 5 — Correção dos problemas

Com o comando direto de corrigir e entregar os arquivos, a IA aplicou as cinco correções de forma cirúrgica, verificou cada mudança e entregou o ZIP final corrigido.

4. Resumo dos Prompts Utilizados

#	Tipo de instrução	Resultado obtido
1	Envio de proposta PDF com contexto completo	Base conceitual alinhada ao projeto
2	Análise do código base (ZIP)	Compreensão da arquitetura existente
3	Especificação de 10 requisitos detalhados	Plano + código completo gerado
4	Revisão de bugs (sem modificar)	5 problemas identificados e explicados
5	Correção dos bugs identificados	Arquivos corrigidos e ZIP entregue

5. Observações sobre o Processo

O diferencial desta abordagem foi o **contexto rico fornecido desde o início**: ao invés de pedir código diretamente, o aluno entregou à IA uma proposta estruturada com motivação, fluxo de uso e escolhas técnicas já pensadas. Isso resultou em respostas mais precisas e alinhadas. Em nenhum momento o código foi copiado sem compreensão — cada trecho foi revisado e, quando necessário, ajustado. A IA foi usada como **ferramenta de aceleração**, enquanto as decisões de arquitetura e requisitos partiram integralmente do aluno.